Rapport final — Projet 1

Solveur de Nash $m \times n$ et duopoles (Cournot, Bertrand)

Groupe 1 : ACHTOUK Fatine, ADIL Youness, BELHIMER Ikram, BOULHAIL Hiba
Cycle Ingénieur — Informatique Décisionnelle et IA (IDIA3)
Module : Théorie des Jeux et Systèmes Multi-Agents — ENSA Safi — Pr. A. JRAIFI

Année universitaire 2026–2027

Table des matières

1 Introduction et problématique **2**

2 Organisation et planification **2**
2.1 Membres et rôles (semaines 1 à 4) . . . 2
2.2 Gantt prévisionnel (déposé en S1) . . . 2
2.3 Gantt réalisé et analyse des écarts . . . 2

3 Théorie : formules clés, hypothèses et limites **2**
3.1 Équilibre de Nash et indifférence sur les supports . . . 2
3.2 Cournot et Bertrand . . . 3
3.3 Hypothèses et limites . . . 3

4 Méthodologie **3**
4.1 Fonctions pures (`src/jeux.py`) . . . 3
4.2 Tests (`tests/`, pytest) . . . 4
4.3 Outils agent (LangChain / Ollama) . . . 4

5 Résultats des expériences et discussion **5**
5.1 Énumération des supports : nombre d'équilibres et temps de calcul . . . 5
5.2 Agent : traduction des énoncés en appels d'outils . . . 5
5.3 Analyse des écarts à la théorie . . . 6

6 Conclusion et lien avec le cours **6**

A Reproductibilité **7**

1 Introduction et problématique

Le cours de Théorie des Jeux et Systèmes Multi-Agents introduit un solveur `nash_mixte_2x2` limité aux jeux 2×2. Or la plupart des situations économiques réelles (oligopoles, enchères, négociations) se modélisent par des jeux à plus de deux stratégies. **Problématique :** comment généraliser le calcul des équilibres de Nash à des jeux finis $m \times n$ quelconques, puis mettre ce calcul à disposition d'un agent conversationnel (LLM local) qui traduit un énoncé en langage naturel en un appel d'outil correct ?

Le projet répond en trois étapes : (i) un solveur exact par *énumération des supports*, (ii) son application aux duopoles de **Cournot** et de **Bertrand**, (iii) son exposition à l'agent LangChain du cours connecté au LLM local `llama3.1` via Ollama. Le rapport décrit ensuite l'organisation, la théorie, la méthodologie, les résultats expérimentaux et les écarts observés.

Le projet consiste à généraliser le solveur `nash_mixte_2x2` du cours en un solveur d'équilibres de Nash pour jeux à deux joueurs de taille quelconque $m \times n$ par *énumération des supports*, puis à l'appliquer aux duopoles de **Cournot** et de **Bertrand** et à l'exposer à l'agent LangChain du cours (LLM local Ollama).

2 Organisation et planification

2.1 Membres et rôles (semaines 1 à 4)

Membre	Rôle (S1–S4)	Rôle (S5–S7, après rotation)
ACHTOUK Fatine	Chef de projet / Théorie	Testeur / Expérimentateur
ADIL Youness	Développeur agent	Chef de projet / Théorie
BELHIMER Ikram	Quant / Python	Développeur agent
BOULHAIL Hiba	Testeur / Expérimentateur	Quant / Python

2.2 Gantt prévisionnel (déposé en S1)

2.3 Gantt réalisé et analyse des écarts

Analyse des écarts. Les phases 0 à 4 (cadrage, théorie, données, code Python, outils agent) sont *terminées* et non plus seulement planifiées : le code des jeux du cours est reproduit, les tests passent et l'agent LangChain réel fonctionne déjà, alors que le plan situait l'agent en S4. Autrement dit, la partie technique a été *avancée* par rapport au prévisionnel, ce qui a dégagé du temps pour la phase d'expériences (S5). Seules les phases 5 (expériences agents) et 6–7 (rédaction, bilan) restent en cours ou à faire. Aucun retard bloquant n'est à signaler ; la seule difficulté d'environnement (Python 3.15 vs `zstandard`) est documentée en annexe.

3 Théorie : formules clés, hypothèses et limites

3.1 Équilibre de Nash et indifférence sur les supports

Un profil mixte (x, y) est un équilibre de Nash si et seulement si

$$(Ay)_i \le x^\top A y \ \ \forall i, \qquad (x^\top B)_j \le x^\top B y \ \ \forall j.$$

Sur les supports $I = \mathrm{supp}(x)$, $J = \mathrm{supp}(y)$ avec $|I| = |J| = k$:

$$(Ay)_i = v_1 \ (i \in I), \qquad (x^\top B)_j = v_2 \ (j \in J), \qquad \textstyle\sum_{i \in I} x_i = \sum_{j \in J} y_j = 1.$$

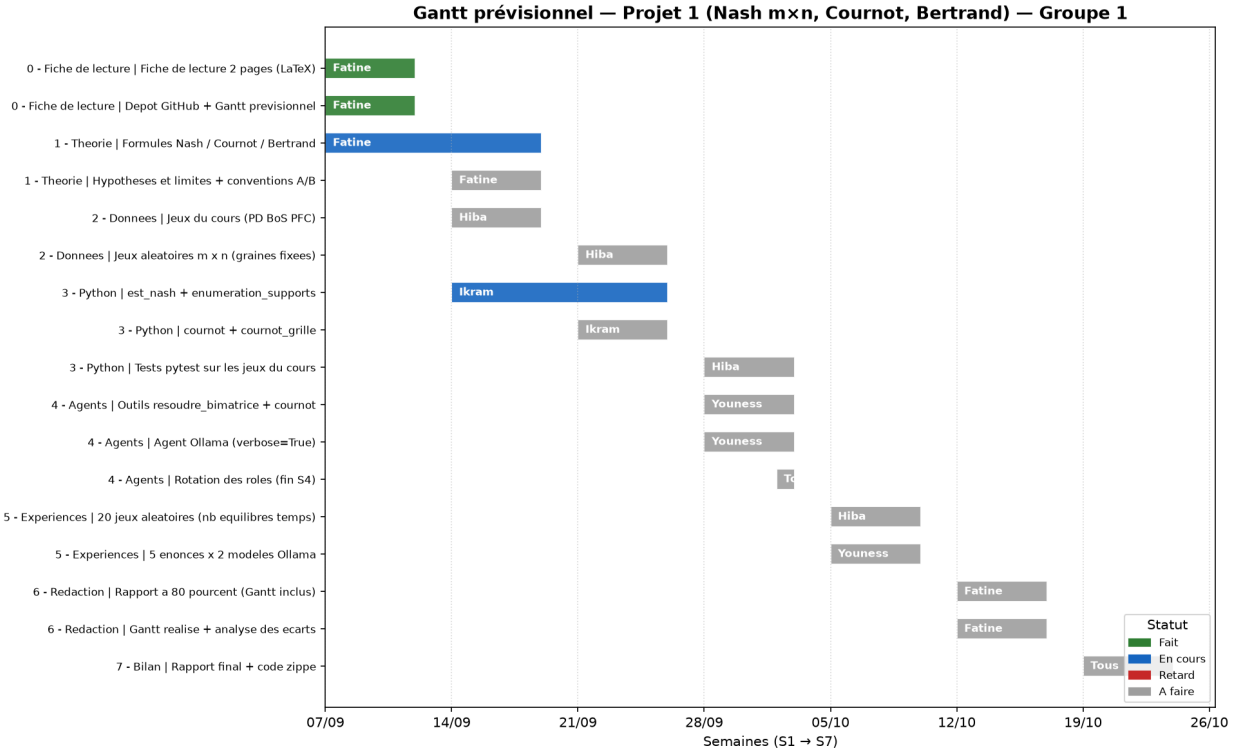


FIGURE 1 – Gantt prévisionnel du projet (7 semaines).

3.2 Cournot et Bertrand

$$q_i = \frac{a - c - b\sum_{j \neq i} q_j}{2b}, \qquad q^* = \frac{a-c}{(n+1)b}, \qquad \pi^* = \frac{(a-c)^2}{b\,(n+1)^2}.$$

Bertrand (biens homogènes, coûts égaux c) : $p_1 = p_2 = c$, profit nul.

3.3 Hypothèses et limites

Hypothèses : jeux statiques finis à deux joueurs, à information complète et mémoire parfaite ; matrices de gains connues ; jeux *non dégénérés* (aucune ligne ou colonne dominée de façon à créer un continuum d'équilibres). Pour Cournot : demande inverse linéaire $P(Q) = a - bQ$ avec $a > c$, coût marginal constant c, biens homogènes, concurrence en quantités et jeu simultané. Pour Bertrand : biens homogènes et coûts identiques.

Limites : (i) l'énumération des supports est de complexité *exponentielle* en la taille du jeu, on plafonne donc à 8×8 ; (ii) les calculs sont numériques, des tolérances (10^{-9}) sont nécessaires pour comparer les gains et déclarer un équilibre ; (iii) le modèle ne traite ni les jeux séquentiels (forme extensive), ni les jeux à information incomplète (Bayésiens) ; (iv) l'agent LLM peut mal traduire un énoncé, ce qui constitue une source d'erreur distincte du solveur (voir §6.3).

4 Méthodologie

4.1 Fonctions pures (`src/jeux.py`)

`est_nash`, `enumeration_supports`, `cournot`, `cournot_grille`, `bertrand`.

Algorithme. Pour chaque couple de supports $I \subseteq \{1..m\}$, $J \subseteq \{1..n\}$ de même cardinal k, on résout le système linéaire d'indifférence $Ay = v_1\mathbf{1}$ sur I, $x^\top B = v_2\mathbf{1}$ sur J, complété par $\sum x_i = \sum y_j = 1$. La solution est retenue si $x \geq 0$, $y \geq 0$ et si aucune stratégie hors support n'est profitable

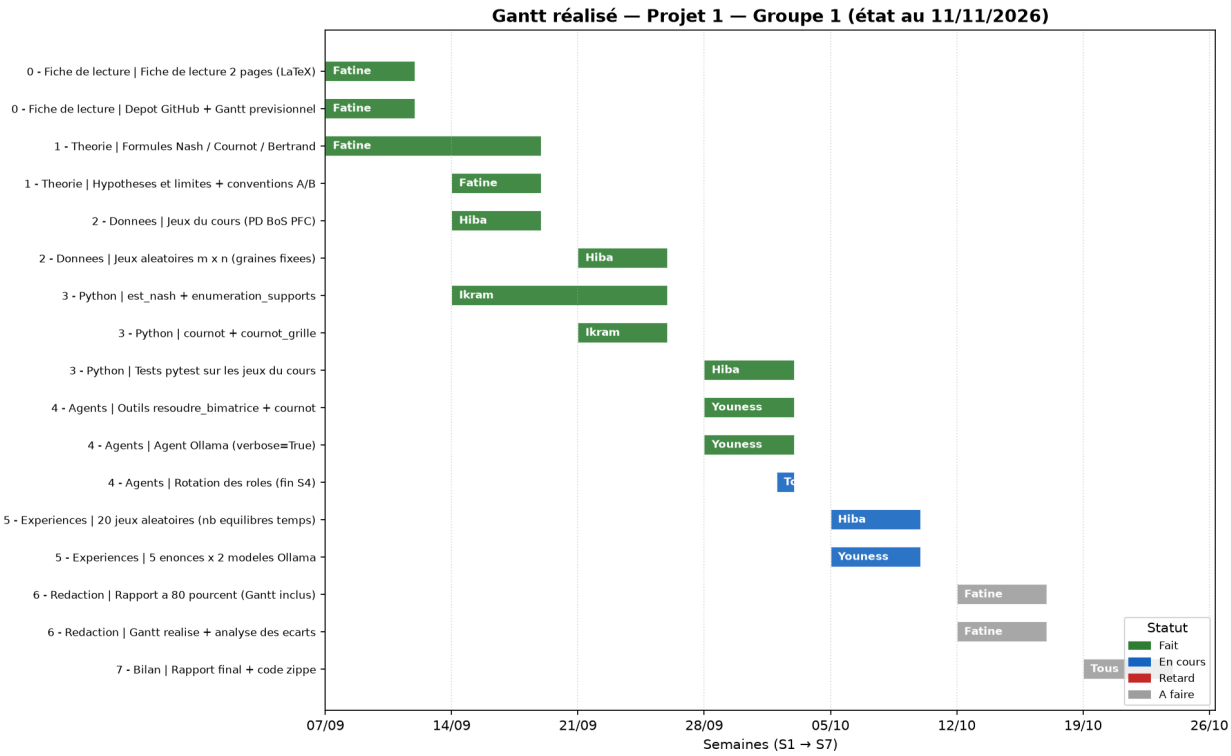


FIGURE 2 – Gantt réalisé (état au 11/11/2026).

(condition d'*équilibre* globale). `est_nash` vérifie a posteriori cette dernière condition, ce qui sert de garde-fou indépendant.

Complexité. Le nombre de couples de supports est $\sum_{k=1}^{\min(m,n)} \binom{m}{k}\binom{n}{k}$, soit une croissance exponentielle : c'est la raison du plafond à 8×8 et de l'expérience de temps de calcul en §6.1.

4.2 Tests (`tests/`, pytest)

25 tests au total. Ils couvrent : (i) les *jeux du cours* — dilemme du prisonnier (1 équilibre pur), bataille des sexes (2 purs + 1 mixte), pierre-feuille-ciseaux ($(1/3, 1/3, 1/3)$) ; (ii) la propriété que *toute* solution renvoyée par `enumeration_supports` passe `est_nash` ; (iii) les cas d'erreur (tailles incompatibles, dépassement de plafond) ; (iv) les *outils sans LLM* (`.invoke`) y compris la robustesse aux matrices passées en chaîne JSON et l'exécution d'un `StructuredTool` LangChain. Les tests ne font aucun appel réseau.

4.3 Outils agent (LangChain / Ollama)

`resoudre_bimatrice(A, B)` et `cournot(a, b, c, n_firmes)`, plafonnés et en JSON. Test `.invoke()` sans LLM (`tests/test_hors_ligne.py`), puis agent Ollama réel (`src/agent_langchain_demo.py`, `create_agent` + `ChatOllama`).

Robustesse aux entrées du LLM. Deux écueils ont été rencontrés et corrigés : (i) `llama3.1` transmet parfois les matrices sous forme de *chaîne* (`"[[3, 0], [5, 1]]"`) ; le validateur pydantic de LangChain rejetait alors l'appel *avant* l'exécution, corrigé par le type `Matrice = Union[list, str]` et un décodage `json.loads` (jamais `eval`, Règle n°5) ; (ii) selon que LangChain est installé ou non, un `@tool` est un `StructuredTool` (non appelable directement) ou une simple fonction, ce que gère `_executer_outil`. Les arguments exacts relevés sur les 5 énoncés sont analysés en §6.2.

5 Résultats des expériences et discussion

5.1 Énumération des supports : nombre d'équilibres et temps de calcul

Protocole : 20 jeux aléatoires par taille ($m = n \in \{2, 3, 4\}$), gains tirés uniformément dans $[-1, 1]$, graines fixées (`np.random.default_rng`) pour la reproductibilité. On relève le nombre d'équilibres trouvés et le temps d'exécution de `enumeration_supports` (tableau 1 et figure 3).

TABLE 1 – Résultats sur 20 jeux aléatoires par taille.

Taille $m = n$	Équilibres (moy.)	Équilibres (max)	Temps moyen (ms)	Temps max (ms)
2	1,10	3	0,90	1,52
3	1,60	3	4,87	8,89
4	2,10	7	15,35	22,76

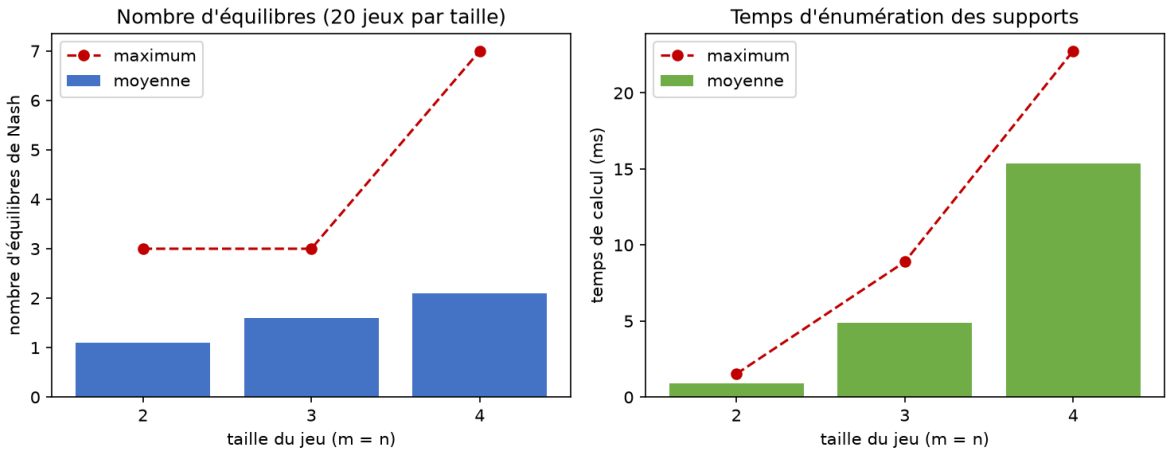


FIGURE 3 – Nombre d'équilibres et temps de calcul en fonction de la taille du jeu.

Lecture. Le nombre d'équilibres croît avec la taille (1,1 en moyenne en 2×2, 2,1 en 4×4) et peut atteindre 7, ce qui confirme qu'un jeu générique admet *plusieurs* équilibres et que la sélection d'équilibre est un vrai problème. Le temps de calcul croît bien plus vite (facteur ≈ 17 entre 2×2 et 4×4), en cohérence avec la complexité exponentielle : à 8×8 le solveur reste utilisable (dixièmes de seconde) mais l'extrapolation est défavorable, d'où le plafond imposé.

5.2 Agent : traduction des énoncés en appels d'outils

Protocole : 5 énoncés × 2 modèles Ollama (`llama3.1` et `llama3.2:3b`), 1 répétition, température 0. Les 5 énoncés comprennent : (1) un Cournot numérique, (2) une bimatrice 2×2, (3) une bimatrice 3×3, (4) une question *conceptuelle* (aucun outil ne doit être appelé), (5) un énoncé *piège* où le coût marginal est donné en pourcentage de l'ordonnée à l'origine. On relève l'outil appelé, la fréquence d'appel et la présence d'une ligne « CHOIX : ».

TABLE 2 – Agent : appels d'outils sur 5 énoncés × 2 modèles.

Modèle	Appels	Outil pertinent	Arguments corrects	Sans « CHOIX : »	Durée moy. (s)
`llama3.1`	5/5	4/5	4/4	5/5	70,1
`llama3.2:3b`	5/5	4/5	3/4	5/5	37,8

« Appels » = énoncés ayant déclenché un appel d'outil (sur 5) ; « Outil pertinent » = outil attendu (4 énoncés sur 5) ; « Arguments corrects » = sur les 4 énoncés à outil ; « Sans CHOIX » = réponses n'explicitant pas la

décision.

Observations. Les deux modèles *appellent un outil* sur les quatre énoncés qui le justifient (Cournot, bimatrices 2×2 et 3×3, énoncé piège) et choisissent l'outil *pertinent* (`cournot` pour les énoncés économiques, `resoudre_bimatrice` pour les jeux matriciels). En revanche, l'examen des *arguments réels* révèle trois types d'écarts :

- **Sur-appel** : sur l'énoncé conceptuel (4), où aucune donnée n'est fournie, les deux modèles appellent quand même un outil. Pire, `llama3.1` *invente* des matrices ($A = [[1,2,3],[4,5,6],[7,8,9]]$) pour satisfaire l'appel : c'est une hallucination de données, l'erreur la plus grave observée.
- **Arguments mal formés, correctement rejetés** : sur le même énoncé, `llama3.2:3b` envoie des listes plates ($A = [1,2,3]$) ; l'outil répond `{"erreur": ...}` au lieu de planter — le garde-fou fonctionne.
- **Erreur de traduction sur l'énoncé piège** : `llama3.1` convertit correctement 20% de 100 en $c = 20$ et obtient $q^* = 26{,}67$; en revanche `llama3.2:3b` introduit une pente fantôme ($b = 0{,}2$) *en plus* de $c = 20$ et renvoie $q^* = 133{,}33$, économiquement absurde. Le modèle 3B échoue donc sur l'énoncé le plus subtil.

Enfin, **aucune** des 10 réponses n'explicite sa décision par une ligne « CHOIX : », ce qui confirme le biais signalé dans le guide : sans relire les arguments des appels d'outils, on ne peut pas savoir ce que le modèle a réellement décidé de faire.

Remarque technique. Le type `Matrice = Union[list, str]` s'est avéré indispensable : `llama3.1` transmet les matrices 2×2 et 3×3 sous forme de *chaînes* JSON, et `cournot` reçoit parfois les scalaires en chaîne (`"100"`, `"20"`) — convertis sans erreur par le validateur.

5.3 Analyse des écarts à la théorie

Trois écarts méritent d'être distingués, car ils n'ont pas la même cause :

1. **Erreur de traduction (modèle).** L'appel d'outil sur l'énoncé conceptuel — et surtout les matrices *inventées* par `llama3.1` — sont des erreurs du LLM, pas du solveur : les mathématiques restent justes, c'est la décision « faut-il calculer, et avec quelles données ? » qui est fausse. Le guide insiste précisément sur ce point. Un énoncé piège mal converti ($b = 0{,}2$ ajouté par le modèle 3B) produit une réponse numériquement cohérente mais *économiquement absurde* ($q^* = 133 > a/b$) : la vérification par un humain reste indispensable.
2. **Robustesse du solveur vs faiblesse du modèle.** Le solveur, lui, ne se trompe pas : il rejette proprement les entrées mal formées et renvoie toujours un JSON exploitable. Les écarts observés proviennent donc de l'interface langage naturel, ce qui est une conclusion importante : *la qualité du résultat dépend surtout de la traduction de l'énoncé*, pas du calcul.
3. **Variabilité d'exécution.** Les temps de réponse varient fortement (20 à 96 s) et un LLM n'est pas déterministe en pratique ; *aucune conclusion ne peut être tirée d'une seule exécution.* C'est pourquoi le protocole fixe la graine des jeux et la température du modèle, et pourquoi il faudrait répéter chaque énoncé plusieurs fois pour estimer un taux d'erreur fiable (limite assumée de cette première campagne).

Bilan. Sur cette campagne, `llama3.1` traduit correctement les énoncés numériques mais hallucine sur l'énoncé sans données ; `llama3.2:3b` est plus rapide mais échoue sur l'énoncé piège. Aucun des deux n'explicite sa décision. Ces résultats plaident pour (i) un énoncé explicite dans le prompt système (« n'appelle un outil que si des données numériques sont fournies ») et (ii) une répétition systématique des essais.

6 Conclusion et lien avec le cours

Le projet atteint ses objectifs : le solveur 2×2 du cours a été généralisé en un solveur exact $m \times n$ par énumération des supports, appliqué à Cournot et Bertrand, et exposé à l'agent LangChain connecté au LLM local `llama3.1`. Les résultats du cours sont reproduits exactement (dilemme, bataille des sexes, pierre-feuille-ciseaux, $q^* = (a - c)/3b$), et 25 tests automatisés garantissent la non-régression.

Apports pédagogiques. Le projet relie trois chapitres du cours : l'équilibre de Nash (indifférence sur les supports), les duopoles (meilleure réponse, comparaison Cournot/cartel) et les systèmes multi-agents (outils, traduction d'un énoncé en appel de fonction). Il met aussi en évidence une limite pratique : un LLM peut produire une *traduction fautive* d'un énoncé correct, erreur qu'aucun test du solveur ne peut détecter.

Prolongements : élimination itérative des stratégies dominées pour réduire la combinatoire, gestion des jeux dégénérés (continuum d'équilibres), extension aux jeux séquentiels et bayésiens, et évaluation systématique de l'agent sur un jeu d'énoncés plus large.

A Reproductibilité

Environnement : **Python 3.12 ou 3.13** (sur Python 3.15, `zstandard` n'a pas de wheel et l'installation de LangChain échoue sans Visual C++ Build Tools) ; `py -3.12 -m venv .venv` puis `pip install -r requirements.txt`. Graines fixées (`np.random.default_rng`) ; commandes : `pytest -q`, `python src/jeux.py`, `python -m src.agent_langchain_demo` (ou `python -m src.agent_demo_ollama` sans LangChain). Expériences : `python docs/rapport/generate_figures.py` (jeux aléatoires) et `python -c "from src.experiences import protocole_agent_ollama as p; p()"` (agent, nécessite `ollama serve`). Données brutes : `exp_jeux.csv`, `exp_agent.csv`.